

Solution 1. The user should provide $S \subseteq V(D)$, such that $\delta^{\text{out}}(S) = B$ and $\delta^{\text{in}}(S) = \emptyset$, where B is the directed cut. This object must be easy to have when the user provides a directed cut.

Algorithm 1 Verify-Directed-Cut(D, B, S)

```

for each  $a \in A(D)$  do
   $v, u \leftarrow \text{ends}[a]$ 
  if  $v \in S$  and  $u \notin S$  then    ▷ This can be done in constant time using an auxiliary array
    if  $a \notin B$  then              ▷ Same thing from above comment
      return False
    end if
  else if  $v \notin S$  and  $u \in S$  then
    return False
  else if  $v \in S$  and  $u \in S$  then
    if  $a \in B$  then
      return False
    end if
  else
    if  $a \in B$  then
      return False
    end if
  end if
end for
return True

```

Solution 2. I will prove that $H_1^{\min} = H_2^{\min}$ by showing that $H_1^{\min} \subseteq H_2^{\min}$ and that $H_2^{\min} \subseteq H_1^{\min}$. We will show that $x \subseteq y$ by proving that an arbitrary element $x_0 \in x$ lies in y .

Given $h_1 \in H_1^{\min}$, h_1 is a minimal rs -cut. By definition, h_1 is a set of all the outwards arcs from a set of vertices that includes r and exclude s , and therefore $h_1 \subseteq H_2$, because it is impossible to have an rs -path if there are no arcs from this set and the other vertices of $V(D)$. h_1 is minimal in H_2 if every element in h_1 is an arc that can be in a different rs -path from the other arcs, and it is not an arc that only is in walks that just adds arcs to an rs -path with another arc in h_1 , otherwise it could be removed, and this new set would be a proper subset of h_1 . First, h_1 does not have more than one arc sharing only same rs -paths because it would not be minimal in $H_1^{\min}$ given that you could just remove all the arcs besides the one that is further from r and the set would still be an rs -cut (a possible shore is to maintain the previous shore and add all the vertices along the paths till the farthest one). Second, there are no arcs in h_1 that are part of an extension (loops) to an rs -path with another arc in h_1 because you could remove it by having a new shore that adds all the vertices of the loops to a shore associated with the previous rs -cut. Finally, it is not possible to have an arc in h_1 that leads to path that does not reach s because this is not a minimal rs -cut given that you could add all the vertices of those dead end paths to the previous shore to have a new shore associated with an rs -cut that is a subset of the previous rs -cut. Given all the arguments, $h_1 \subseteq H_2$, and it is minimal in H_2 , thus $H_1^{\min} \subseteq H_2^{\min}$.

Given $h_2 \in H_2^{\min}$, h_2 is a minimal subset $B \subseteq A$ such that $D - B$ has no rs -path. This is an rs -cut because you can have a shore that is a set of all the vertices connected from r to the vertices that the outward edges are on B . If those vertices have arcs that are not in B , it means that is an arc that does not lead to a new path to s or is part of a loop that goes back to a vertex in the path which the arc that is in B is part of. In both cases, because of the aspects I explained in the last paragraph, you could have a new shore, including all the vertices of the loop and the paths that do not reach s , which is associated with a new rs -cut without those arcs that are not in B . Thus, $h_2 \subseteq H_1$. Also, h_2 is minimal in H_1 because if you remove an arc a_2 from h_2 , you can not have a shore associated with the rs -cut $h_2 - a_2$. This holds because this arc is a unique arc from an rs -path in h_2 . If you add any neighbor of the tail of a_2 in a h_2 shore to try to remove a_2 from h_2 , you will add at least one arc in h_2 because the end of the rs -path is s and s can not be in a shore. If you try to remove vertices from a shore associated with h_2 you would add at least one arc to h_2 because the beginning of the rs -path is r and r is in every possible shore. In both cases you can not remove an arc from h_2 without adding another arc. Given that, $h_2 \subseteq H_1$, and it is minimal in H_1 , thus $H_2^{\min} \subseteq H_1^{\min}$.

By the arguments shown, $H_1^{\min} = H_2^{\min}$.

Solution 3. The following algorithm solves the problem.

Algorithm 2 MinimizeWalk(D, r, s)

```

 $V_{\text{aux}} \leftarrow \emptyset$  ▷ set of vertices of an auxiliary digraph
 $A_{\text{aux}} \leftarrow \emptyset$  ▷ set of arcs of an auxiliary digraph
for each  $v \in V(D)$  do
     $V_{\text{aux}} \leftarrow V_{\text{aux}} \cup v_0 \cup v_1 \cup v_2$ 
end for
for each  $a \in V(A)$  do
     $v, u \leftarrow \text{ends}[a]$ 
     $A_{\text{aux}} \leftarrow A_{\text{aux}} \cup v_0u_1 \cup v_1u_2 \cup v_2u_0$ 
end for
 $\pi, d \leftarrow \text{Breadth-First-Search}((V_{\text{aux}}, A_{\text{aux}}), v)$ 
if  $d[s_1] < \infty$  then
     $p_{\text{aux}} \leftarrow \text{Path-From-Branching}((\pi, d), s)$ 
     $p \leftarrow \text{Relabeling}(p_{\text{aux}})$  ▷ This function will be explained below
    return  $p$ 
end if
return NIL

```

Using the TA's hint, let $D_{\text{aux}}(V_{\text{aux}}, A_{\text{aux}})$ be an auxiliary digraph built from D based on two rules:

- For each $v \in V(D)$, V_{aux} has three vertices v_0, v_1, v_2 , so the size of V_{aux} is three times the size of $V(D)$.
- For each arc $vu \in A(D)$, A_{aux} has three arcs v_0u_1, v_1u_2, v_2u_0 . Also, A_{aux} is three times the size of $A(D)$.

The Relabeling(p_{aux}) function receives a path on the auxiliary digraph and relabel all the vertices and edges to match with the original graph. If a vertex has label $v_i, i \in 0, 1, 2$ it will be relabeled to v . An edge from $v_iu_j, i \in 0, 1, 2, j \in 0, 1, 2$, will be relabeled to vu . This will make sure the path is the according path in the original graph.

After building this auxiliary digraph, we run a Breadth-First-Search and verify if there is a path from r_0 to s_1 . The idea to do that is that every path that ends on a node labeled with the number 1 is a path where the length $l \equiv 1 \pmod{3}$. This is evident because using the auxiliary digraph, every node that you transverse you go from label 0 to 1, 1 to 2, or 2 to 0, meaning that if you start at a vertex with label 0 (r_0), each vertex that you traverse, you are adding a label mod 3, and therefore the length of the path is you find the label mod 3.

Also, if there is a rs walk, where $l \equiv 1 \pmod{3}$, this algorithm will find it. First, every possible walk is also possible in the auxiliary digraph given that every node that is reachable from a node on the original graph is also reachable on the auxiliary digraph. This algorithm account for paths that you go through the same vertex multiple times given that if the rest of the division by three is different for the same thing on the last time you traverse this vertex. So if there is a walk $l \equiv 1 \pmod{3}$, it will be a different path on the new digraph, and therefore the BFS will get this path.

Solution 4. First, it is impossible to have both objects at the same time. If you have a matching that saturates A , every element of A has a distinct neighbor in B by the definition of matching. Therefore, if there is a matching that saturates A , the inequality for $S \subseteq A$, $|S| \leq |N(S)|$ holds.

Algorithm 3 Matching-or-Hall-Set ($G, (A, B)$)

```

1:  $M, K \leftarrow$  Konig's-Algorithm ( $G, (A, B)$ )
2: if  $M$  saturates  $A$  then
3:   return  $M$  ▷ Can I do this?
4: end if
5: for each  $v \in A$  do
6:   if  $v \notin M$  then
7:      $S \leftarrow \emptyset$  ▷ S will be the subset described in the question
8:      $\pi, d \leftarrow$  Breadth-First-Search( $G, v$ )
9:     for each  $u \in V(G)$  do
10:      if ( $d[u] < \infty$ ) and ( $u \in A$ ) then
11:         $S \leftarrow S \cup u$ 
12:      end if
13:    end for
14:    return  $S$ 
15:   end if
16: end for

```

It is correct because if the connected component always go back to the set A if it goes to the set B , because v and the subsequent are only connected to vertices on the matching, therefore this vertex is connected to a vertex in A .

If the vertex has a neighbor it will go back to the set A because if it was not the case the matching would not be the maximum matching.

On line 1 we run Konig's algorithm that was analyzed in class. Lines 2, 3 and 4 can be done in linear time using an array to mark each vertex in A that the arcs on the matching has an end and then go through this array to verify M saturates A . The FOR loop on line 5 runs at most $|A|$ times, but the IF on line 6 is true only one time, because if it is true, latter will have a RETURN at line 14. Given that, we will run one BFS ($O(|V(G)| + |E(G)|)$) and the FOR at line 9 will run one time ($O(|V(G)|)$). Thus, the time of Konig's algorithm defines the time of the algorithm 3, ($O(|V(G)|(|V(G)| + |E(G)|))$).